

```
1  """prelude.py -- Haskell-style Prelude in pure Python, in pedagogical order.
2
3  Pascal discipline: strict define-before-use. Read top to bottom and every name is already defined
4  when you meet it. Sections build on each other:
5
6      1 combinators          2 pairs          3 arithmetic
7      4 list basics          5 higher-order lists  6 folds
8      7 scans                8 streams (lazy)  9 slicing & spans
9      10 sorting & searching 11 strings        12 containers
10     13 nodes              14 grids & windows  15 control
11
12  Within a section names run alphabetically, except that a definition is hoisted ahead of the first
13  name that uses it (foldl before compose, span before break_), so the discipline holds inside
14  sections too.
15
16  Import qualified (import prelude as P): Haskell names shadow builtins by design. Beyond the Prelude
17  proper: the working parts of Data.List, Data.Map, Data.Maybe, Data.Function and friends are folded
18  into the same sections -- grouped by what they do, not by the module they came from. None is
19  Nothing throughout. foldr = foldl(flip(f), reversed(xs), base): iterative, stack-safe, valid only
20  because Python is strict.
21  """
22
23  # ===== 1. combinators ===== (functions about functions)
24
25  const      = lambda x: lambda _: x                # always returns x, ignoring the second argument
26  # curry: converts a two-argument function into a chain of two one-argument functions, so instead
27  # of calling f(x, y), you call curry(f)(x)(y).
28  curry      = lambda f: lambda x: lambda y: f(x, y)
29  flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
30  fromMaybe = lambda d, x: d if x is None else x     # Data.Maybe: default for every None-returning tool
31  id         = lambda x: x                          # shadows builtin id() by design
32  isJust     = lambda x: x is not None               # Data.Maybe: None test as a handable predicate
33  isNothing  = lambda x: x is None                  # Data.Maybe: true when x is Nothing (None)
34  NOTHING   = object()                             # Maybe's Nothing: "no arg given"; test with 'is'
35  on         = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp 'on' fst
36  partial    = lambda f, *bound: lambda *args: f(*bound, *args) # fix leading args of f, wait for the rest
37  uncurry    = lambda f: lambda p: f(*p)            # inverse of curry: call f with a pair, not two args
38
39  # ===== 2. pairs =====
40
41  fst = lambda p: p[0]                                # first element of a pair
42  snd = lambda p: p[1]                                # second element of a pair
43  swap = lambda p: (p[1], p[0])                       # Data.Tuple
44
45  # ===== 3. arithmetic & logic =====
46
47  div        = lambda a, b: a // b                    # floors, like Haskell div
48  even       = lambda n: n % 2 == 0                  # true for even numbers
49  gcd        = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) # >= 0 like Haskell; depth <= 90
50  halve      = lambda n: n // 2                      # e.g. iterate(halve, 1024) -> 1024, 512, 256, ...
51  hypot      = lambda x, y: (x*x + y*y) ** 0.5        # straight-line distance
52
53  def isqrt(n): # floor sqrt without floats (Newton)
54      if n < 0:
55          raise ValueError("isqrt of negative")
56      x = n
57      y = halve(x + 1)
58      while y < x:
59          x, y = y, halve(y + n // y)
60      return x if n else 0
61
62  lcm        = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like Haskell
63  mod        = lambda a, b: a % b                      # remainder, sign follows b
64  not_       = lambda b: not b                         # 'not' is a Python keyword
65  odd        = lambda n: n % 2 == 1                   # true for odd numbers
66  otherwise  = True                                    # guard fallback: always true
67  pred       = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
68  quot       = lambda a, b: -(a // b) if (a < 0) != (b < 0) else a // b   # truncates, like Haskell quot
69  rem        = lambda a, b: a - b * quot(a, b)          # remainder, sign follows a
70  quotRem    = lambda a, b: (quot(a, b), rem(a, b))     # (quot, rem) in one call
71  signum     = lambda x: (x > 0) - (x < 0)              # -1, 0, or 1 by the sign of x
72  succ       = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1 # next: increments, advances chars
73
74  # ===== 4. list basics =====
75
76  drop       = lambda n, xs: list(xs)[max(n, 0):]      # finite lists; n<0 drops none
77  elem       = lambda x, xs: x in xs                  # true if x is in xs
78  zip_       = lambda a, b: list(zip(a, b))            # shortest wins, any iterable
79  enum       = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x with its in
80  dex
81  head       = lambda xs: xs[0]                      # first element
82  init       = lambda xs: xs[:-1]                    # all but the last element
83  isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p) # Data.List; strings or lists
84  isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
85  last       = lambda xs: xs[-1]                    # last element
86  lookup     = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
87  notElem    = lambda x, xs: x not in xs              # true if x is not in xs
88  nub        = lambda xs: list(dict.fromkeys(xs))      # unique; first seen wins
```

```
88 null = lambda xs: len(xs) == 0 # true for an empty list
89 pairwise = lambda xs: list(zip(xs, xs[1:])) # zip xs (tail xs)
90 replicate = lambda n, x: [x] * n # x repeated n times
91 splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front
92 stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None # Just the rest, or Nothing
93 tail = lambda xs: xs[1:] # all but the first element
94 def transpose(xss): # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
95     xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot); materialise once
96     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
97     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
98 unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], []) # inverse of zip: pairs -> two l
99 lists = lambda a, b, c: list(zip(a, b, c)) # zip three lists together
100
101 # ===== 5. higher-order lists =====
102
103 filter_ = lambda crit, xs: [x for x in xs if crit(x)] # keep elements where crit holds
104 catMaybes = partial(filter_, isJust) # mapMaybe id
105 concat = lambda xss: [x for xs in xss for x in xs] # flatten one level of nesting
106 concatMap = lambda f, xs: [y for x in xs for y in f(x)] # map f over xs, then flatten
107 cross = lambda a, b: [(x, y) for x in a for y in b] # cartesian; 'product' is the fold
108 # Data.List find: lazy first-match -- folds can't stop early, find can
109 find = lambda crit, xs: next((x for x in xs if crit(x)), None) # first match, else None
110 map_ = lambda f, xs: [f(x) for x in xs] # apply f to every element
111 # Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter shape
112 mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
113 def partition(crit, xs): # (keepers, rest) in ONE pass; crit called once per element
114     yes, no = [], []
115     for x in xs:
116         (yes if crit(x) else no).append(x)
117     return (yes, no)
118 starmap = lambda f, pairs: [f(*p) for p in pairs] # map . uncurry
119 zipWith = lambda f, a, b: [f(x, y) for x, y in zip(a, b)] # combine two lists elementwise
120 zipWith3 = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists elementwise
121
122 # ===== 6. folds ===== (collapse a list to one value)
123
124 def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
125     it = iter(xs)
126     if base is NOTHING:
127         try:
128             acc = next(it)
129             except StopIteration:
130                 raise TypeError("fold of empty sequence with no initial value")
131     else:
132         acc = base
133     for x in it:
134         acc = f(acc, x)
135     return acc
136
137 foldr = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
138 compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST first
139 foldl1 = lambda f, xs: foldl(f, xs) # foldl seeded by the first element
140 foldr1 = lambda f, xs: foldl(flip(f), reversed(list(xs))) # foldr seeded by the last element
141 product = lambda xs: foldl(lambda a, x: a * x, xs, 1) # the numeric fold
142
143 # enumeration folds -- brute-force licenses for small n (say the bound out loud):
144 # Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
145 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
146 # Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
147 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
148
149 # ===== 7. scans ===== (a fold that keeps its history)
150
151 def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
152     if f is None:
153         f = lambda a, b: a + b
154     it = iter(xs)
155     if initial is NOTHING:
156         try:
157             acc = next(it)
158             except StopIteration:
159                 return
160     else:
161         acc = initial
162     yield acc
163     for x in it:
164         acc = f(acc, x)
165         yield acc
166
167 scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every accumulator
168 scanl1 = lambda f, xs: list(accumulate(xs, f)) # scanl with no explicit base
169 scanr = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs))))) # foldr, keeping ev
170     every acc
171 scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs))))) # scanr with no bas
172
173 # ===== 8. streams ===== (lazy, possibly infinite)
```

```
173
174 chain      = lambda *iterables: (x for it in iterables for x in it)    # concat, lazily
175
176 def count(start=0, step=1):                                           # [start, start+step ..]
177     n = start
178     while True:
179         yield n
180         n += step
181
182 def cycle(xs):                                                         # repeat xs forever
183     xs = list(xs)
184     if not xs:
185         raise ValueError("cycle: empty list")    # Haskell errors too; the alternative is a silent hang
186     while True:
187         yield from xs
188
189 def iterate(f, x):                                                    # iterate f x = [x, f x, f (f x), ..]
190     while True:
191         yield x
192         x = f(x)
193
194 def repeat(x, n=None):                                                # yield x forever, or n times if given
195     if n is None:
196         while True:
197             yield x
198     else:
199         for _ in range(n):
200             yield x
201
202 def unfoldr(f, seed):                                                 # Data.List unfoldr: f(seed) -> None | (value, seed')
203     out = []
204     step = f(seed)
205     while step is not None:
206         val, seed = step
207         out.append(val)
208         step = f(seed)
209     return out
210
211 # ===== 9. slicing & spans ===== (finite views of streams)
212
213 def span(p, xs):                                                      # split at first failure, ONE pass; safe on one-shot iters
214     xs = list(xs)
215     i = next((i for i, x in enumerate(xs) if not p(x)), len(xs))
216     return (xs[:i], xs[i:])
217 break_      = lambda p, xs: span(lambda x: not p(x), xs)              # split at first hit (opposite of span)
218
219 chunksOf    = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short last ok
220
221 def dropwhile(crit, xs):                                              # generator: drop the leading run where crit holds
222     it = iter(xs)
223     for x in it:
224         if not crit(x):
225             yield x
226             break
227     yield from it
228
229 dropWhile   = lambda p, xs: list(dropwhile(p, xs))                  # drop the leading run where p holds, as a list
230 inits       = lambda xs: [xs[:i] for i in range(len(xs) + 1)]       # Data.List: every prefix, [] first
231
232 def islice(iterable, stop):                                           # take
233     it = iter(iterable)
234     for _ in range(stop):
235         try:
236             yield next(it)
237         except StopIteration:
238             return
239
240 tails       = lambda xs: [xs[i:] for i in range(len(xs) + 1)]       # every suffix; substrings start here
241 take        = lambda n, xs: list(islice(iter(xs), n))               # works on infinite streams
242
243 def takeuntil(crit, xs):                                              # like takewhile(not . crit), but INCLUDES the first hit
244     for x in xs:
245         yield x
246         if crit(x):
247             return
248
249 def takewhile(crit, xs):                                              # generator: yield the leading run where crit holds
250     for x in xs:
251         if not crit(x):
252             return
253         yield x
254
255 takeWhile   = lambda p, xs: list(takewhile(p, xs))                  # take the leading run where p holds, as a list
256
257 # ===== 10. sorting & searching =====
258
259 def bisect_left(a, x):                                                # first index where a[i] >= x
260     lo, hi = 0, len(a)
```

```
260     while lo < hi:
261         mid = halve(lo + hi)
262         if a[mid] < x:
263             lo = mid + 1
264         else:
265             hi = mid
266     return lo
267
268 def bisect_right(a, x):                # first index where a[i] > x
269     lo, hi = 0, len(a)
270     while lo < hi:
271         mid = halve(lo + hi)
272         if a[mid] <= x:
273             lo = mid + 1
274         else:
275             hi = mid
276     return lo
277
278 def cmp_to_key(cmp):                  # resurrect Python 2's cmp for sortBy
279     class K:
280         def __init__(self, x): self.x = x
281         def __lt__(self, other): return cmp(self.x, other.x) < 0
282     return K
283
284 maxOn = lambda f, xs: max(xs, key=f)    # maximumBy (comparing f) in O(n); first wins ties
285
286 def merge(a, b):                      # merge two sorted lists, stable; the heart of merge sort
287     out, i, j = [], 0, 0                # (section 13's merge_lists is this same loop on ListNodes)
288     while i < len(a) and j < len(b):
289         if a[i] <= b[j]:
290             out.append(a[i]); i += 1
291         else:
292             out.append(b[j]); j += 1
293     return out + a[i:] + b[j:]
294
295 minOn = lambda f, xs: min(xs, key=f)    # minimumBy (comparing f): sortOn + head without the sort
296 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))    # comparator sort, Python 2 style
297 sortOn = lambda f, xs: sorted(xs, key=f)    # sortOn (schwartzian, f called once per element)
298
299 # ===== 11. strings =====
300
301 lines = str.splitlines                # split on newlines
302 unlines = lambda ls: "".join(l + "\n" for l in ls)    # join lines back with trailing newlines
303 unwords = " ".join                    # join words with single spaces
304 words = str.split                      # split on whitespace
305
306 # ===== 12. containers =====
307
308 # multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
309 bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)}    # clipped at 0
310 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k, 0)))}    # min
311                                     # count per shared key
312 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())    # a <= b (inclusion)
313 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
314
315 class defaultdict(dict):              # dict that manufactures a value via factory() on first miss
316     def __init__(self, factory=None, *args, **kw):
317         super().__init__(*args, **kw)
318         self.factory = factory
319     def __missing__(self, key):
320         if self.factory is None:
321             raise KeyError(key)
322         self[key] = self.factory()
323         return self[key]
324
325 def Counter(xs):                      # count-by-value; a defaultdict(int) fold
326     d = defaultdict(int)
327     for x in xs:
328         d[x] += 1
329     return d
330
331 class deque:                          # two-stack; all four ends amortized O(1)
332     def __init__(self, xs=()):
333         self._in, self._out = [], list(reversed(list(xs)))
334     def append(self, x): self._in.append(x)
335     def appendleft(self, x): self._out.append(x)
336     def pop(self):
337         if not self._in:
338             self._in, self._out = list(reversed(self._out)), []
339         return self._in.pop()
340     def popleft(self):
341         if not self._out:
342             self._out, self._in = list(reversed(self._in)), []
343         return self._out.pop()
344     def __len__(self): return len(self._in) + len(self._out)
345     def __iter__(self): return iter(self._out[::-1] + self._in)
346
347 def dsu(n):                           # union-find, path halving; union -> False if already joined
```

```
347 parent = list(range(n))
348 def find(x):
349     while parent[x] != x:
350         parent[x] = parent[parent[x]]
351         x = parent[x]
352     return x
353 def union(a, b):
354     ra, rb = find(a), find(b)
355     if ra == rb:
356         return False
357     parent[ra] = rb
358     return True
359 return find, union
360
361 def fromListWith(f, pairs):
362     d = {}
363     for k, v in pairs:
364         d[k] = f(v, d[k]) if k in d else v
365     return d
366     # Counter == fromListWith(add) over (x, 1);
367     # grouping == fromListWith(++) over (k, [v])
368
369 def getpath(t, ks, default=None):
370     for k in ks:
371         if not isinstance(t, dict) or k not in t:
372             return default
373         t = t[k]
374     return t
375
376 def groupBy(eq, xs):
377     out = []
378     for x in xs:
379         if out and eq(out[-1][0], x):
380             out[-1].append(x)
381         else:
382             out.append([x])
383     return out
384     # Data.List groupBy: runs of CONSECUTIVE elements; each new
385     # element is compared via eq against the run's FIRST element,
386     # exactly like Haskell (span-based) -- not its neighbor
387
388 group = lambda xs: groupBy(lambda a, b: a == b, xs) # Data.List group: runs of equals, [[a]] -- no keys
389
390 def heappop(h):
391     h[0], h[-1] = h[-1], h[0]
392     top = h.pop()
393     i, n = 0, len(h)
394     while True:
395         s, l, r = i, 2*i + 1, 2*i + 2
396         if l < n and h[l] < h[s]: s = l
397         if r < n and h[r] < h[s]: s = r
398         if s == i:
399             return top
400         h[i], h[s] = h[s], h[i]
401         i = s
402
403 def heappush(h, x):
404     h.append(x)
405     i = len(h) - 1
406     while i and h[halve(i - 1)] > h[i]:
407         h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
408         i = halve(i - 1)
409
410 insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map insertWith: PERSISTENT
411                                     # (fresh dict); f(new, old)
412 ITree = lambda: defaultdict(ITree) # infinitely-nestable defaultdict: a tree that grows on access
413
414 def paths(t):
415     # cata for Tree/ITree: [(keypath, leaf)]
416     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
417     if not isinstance(t, dict) or not t: # leaf = non-dict value, or empty node
418         return [([], t)]
419     return [([k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]
420
421 leaves = lambda t: [l for _, l in paths(t)] # every leaf value in a Tree/ITree, in path order
422
423 def setpath(t, ks, v):
424     for k in ks[:-1]:
425         t = t[k]
426     t[ks[-1]] = v
427     # NB: clobbers any subtree already at ks
428
429 Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1 # fixed-depth nested defaultdict;
430                             else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default to leaf()
431
432 unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
433     # Data.Map unionWith: f(a's value, b's value) on shared keys;
434     # bag_union == unionWith(max), merge-sum == unionWith(add)
435
436 # ===== 13. nodes ===== (the LeetCode gives: binary trees & linked lists)
437
438 class ListNode:
439     # the LeetCode linked list
440     def __init__(self, val=0, nxt=None):
441         self.val, self.next = val, nxt
```

```
435
436 class TreeNode:                                # the LeetCode binary tree
437     def __init__(self, val=0, left=None, right=None):
438         self.val, self.left, self.right = val, left, right
439
440 from_list = lambda xs: foldr(ListNode, xs, None)    # build; foldr of ListNode
441
442 def has_cycle(node):                             # Floyd: fast laps slow iff a cycle exists
443     slow = fast = node
444     while fast and fast.next:
445         slow, fast = slow.next, fast.next.next
446         if slow is fast:
447             return True
448     return False
449
450 # tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth = tree height
451 tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
452 inorder = lambda t: tfold(lambda v, l, r: l + [v] + r, [], t)    # left, root, right
453
454 def levelorder(t):                                # BFS: the deque earning its keep
455     out, q = [], deque([t] if t else [])
456     while len(q):
457         n = q.popleft()
458         out.append(n.val)
459         if n.left: q.append(n.left)
460         if n.right: q.append(n.right)
461     return out
462
463 def merge_lists(a, b):                            # merge two sorted lists; dummy-head idiom
464     dummy = tail_ = ListNode()
465     while a and b:
466         if a.val <= b.val:
467             tail_.next, a = a, a.next
468         else:
469             tail_.next, b = b, b.next
470         tail_ = tail_.next
471     tail_.next = a or b
472     return dummy.next
473
474 def middle(node):                                  # fast/slow: second middle for even lengths
475     slow = fast = node
476     while fast and fast.next:
477         slow, fast = slow.next, fast.next.next
478     return slow
479
480 postorder = lambda t: tfold(lambda v, l, r: l + r + [v], [], t)    # left, right, root
481 preorder = lambda t: tfold(lambda v, l, r: [v] + l + r, [], t)    # root, left, right
482
483 def reverse_list(node):                            # three-pointer; prev is a fold accumulator
484     prev = None
485     while node:
486         node.next, prev, node = prev, node, node.next
487     return prev
488
489 tdepth = lambda t: tfold(lambda _, l, r: l + max(l, r), 0, t)    # height of the tree
490 to_list = lambda node: list(unfoldr(lambda n: None if n is None else (n.val, n.next), node)) # ListNode
491 chain -> list
492 tsize = lambda t: tfold(lambda _, l, r: l + 1 + r, 0, t)        # number of nodes in the tree
493
494 # ===== 14. grids & windows =====
495
496 def longest_window(xs, valid, add, shed):           # sliding-window skeleton; state lives in the closures
497     lo = best = 0
498     for hi in range(len(xs)):
499         add(xs[hi])
500         while not valid():
501             shed(xs[lo])
502             lo += 1
503         best = max(best, hi - lo + 1)
504     return best
505
506 def neighbors4(r, c, R, C):                        # in-bounds orthogonal neighbors
507     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
508         if 0 <= r + dr < R and 0 <= c + dc < C:
509             yield r + dr, c + dc
510
511 def neighbors8(r, c, R, C):                        # ... plus diagonals
512     for dr in (-1, 0, 1):
513         for dc in (-1, 0, 1):
514             if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
515                 yield r + dr, c + dc
516
517 # ===== 15. control =====
518
519 def memo(f):                                       # unbounded memoizer; enough for DP (no eviction by design)
520     cache = {}
521     def wrapped(*args, **kw):
522         key = (args, frozenset(kw.items())) if kw else args
```

```
522         if key not in cache:
523             cache[key] = f(*args, **kw)
524         return cache[key]
525     wrapped.cache = cache           # peek at the DP table if curious
526     return wrapped
527
528 def until(p, f, x):                # until p f x ? loop, not recursion (unbounded depth)
529     while not p(x):
530         x = f(x)
531     return x
end
```